ML Assignment: Binary Classification

Predicting Purchase Intention of Online Shoppers (UCI Dataset #468)

Problem type	Binary classification
Dataset	Online Shoppers Purchasing Intention Dataset (UCI Machine Learning Repository)
Models	Logistic Regression · Decision Tree · K-Nearest Neighbors · Gaussian Naive Bayes · Support Vector Machine
Metrics	Accuracy · AUC · Precision · Recall · F1 · MCC

> This report is the executed Jupyter notebook. All the logic it runs (cleaning, preprocessing, model definitions, tuning, evaluation, saving) lives in the project's `src/` package, so the notebook, `train_models.py` and the Streamlit `app.py` share exactly the same code.

Step 1: Problem Statement & Dataset

1.1 Problem statement

An e-commerce website receives thousands of browsing sessions a day, but only about **1 in 6 sessions ends in a purchase**. If the site can tell **during a session** whether the visitor is likely to buy, it can act on that: show a discount or live chat to undecided visitors, retarget likely buyers, or stop spending ad budget on sessions that won't convert.

> **Goal:** Given the clickstream behaviour, Google-Analytics page metrics and visit context of a session, **predict whether the session will end in a purchase** (`Revenue = 1`) **or not** (`Revenue = 0`).

1.2 Dataset

Item	Detail
Name	Online Shoppers Purchasing Intention Dataset
Source	UCI Machine Learning Repository, dataset id **468**: https://archive.ics.uci.edu/dataset/468/online+shoppers+purchasing+intention+dataset
Citation	Sakar, C. & Kastro, Y. (2018). *Online Shoppers Purchasing Intention Dataset*. UCI ML Repository. DOI: 10.24432/C5F88Q
Paper	Sakar, C.O., Polat, S.O., Katircioglu, M., Kastro, Y. (2019). *Real-time prediction of online shoppers' purchasing intention using multilayer perceptron and LSTM recurrent neural networks.* Neural Computing & Applications.
License	CC BY 4.0
File	`data/online_shoppers_intention.csv` (CSV, 18 columns)

1.3 Assignment constraints check

Requirement	Required	This dataset	Status
Public repository	Kaggle / UCI	UCI #468	✅
Problem type	Binary or multi-class	**Binary** (`Revenue`: True/False)	✅
Minimum features	12	**17** input features	✅
Minimum instances	500	**12,330** sessions	✅

1.4 Feature dictionary

#	Feature	Type	Description
1	`Administrative`	Numeric (count)	No. of administrative pages visited (account, login etc.)
2	`Administrative_Duration`	Numeric (sec)	Time spent on administrative pages
3	`Informational`	Numeric (count)	No. of informational pages visited (about, contact, policies)
4	`Informational_Duration`	Numeric (sec)	Time spent on informational pages
5	`ProductRelated`	Numeric (count)	No. of product pages visited
6	`ProductRelated_Duration`	Numeric (sec)	Time spent on product pages
7	`BounceRates`	Numeric (0–0.2)	Avg. bounce rate (Google Analytics) of pages visited
8	`ExitRates`	Numeric (0–0.2)	Avg. exit rate of pages visited
9	`PageValues`	Numeric	Avg. page value of pages visited before a transaction
10	`SpecialDay`	Numeric (0–1)	Closeness of visit date to a special day (Valentine's, Mother's Day …)
11	`Month`	Categorical	Month of the visit (10 months present)
12	`OperatingSystems`	Categorical (coded 1–8)	OS of the visitor
13	`Browser`	Categorical (coded 1–13)	Browser of the visitor
14	`Region`	Categorical (coded 1–9)	Geographic region
15	`TrafficType`	Categorical (coded 1–20)	Traffic source (direct, ad, referral …)
16	`VisitorType`	Categorical	Returning / New / Other visitor
17	`Weekend`	Boolean	Whether the visit was on a weekend
Target	**`Revenue`**	**Boolean**	**Whether the session ended in a purchase**

In [1]:

```
import logging
import sys
from pathlib import Path

# Run from the project folder so the `src` package is importable
PROJECT_ROOT = Path.cwd()
sys.path.insert(0, str(PROJECT_ROOT))

import matplotlib.pyplot as plt
import pandas as pd

from src import config
from src import visualization as viz
from src.data import DataPreparer, load_raw_data, remove_duplicates, split_train_test
from src.evaluation import ModelEvaluator
from src.models import MODEL_SPECS
from src.persistence import ModelRepository
from src.preprocessing import build_preprocessor
from src.training import HyperparameterTuner, silence_expected_warnings
from src.workflow import TrainingWorkflow, save_artifacts

logging.basicConfig(level=logging.WARNING, format="%(message)s", stream=sys.stdout, force=True)
logging.getLogger("src").setLevel(logging.INFO)   # show this project's progress messages only
silence_expected_warnings()
pd.set_option("display.max_colwidth", None)
```

1.5 Load the data

In [2]:
```
raw = load_raw_data(config.DATA_PATH)
print("Shape:", raw.shape)
raw.head().T   # first 5 sessions, transposed so all 18 columns are readable
```

Out [2]:
```
Shape: (12330, 18)
```

	0	1	2	3	4
Administrative	0	0	0	0	0
Administrative_Duration	0.0	0.0	0.0	0.0	0.0
Informational	0	0	0	0	0
Informational_Duration	0.0	0.0	0.0	0.0	0.0
ProductRelated	1	2	1	2	10
ProductRelated_Duration	0.0	64.0	0.0	2.666667	627.5
BounceRates	0.2	0.0	0.2	0.05	0.02
ExitRates	0.2	0.1	0.2	0.14	0.05
PageValues	0.0	0.0	0.0	0.0	0.0
SpecialDay	0.0	0.0	0.0	0.0	0.0
Month	Feb	Feb	Feb	Feb	Feb
OperatingSystems	1	2	4	3	3
Browser	1	2	1	2	3
Region	1	1	9	2	1
TrafficType	1	2	3	4	4
VisitorType	Returning_Visitor	Returning_Visitor	Returning_Visitor	Returning_Visitor	Returning_Visitor
Weekend	False	False	False	False	True
Revenue	False	False	False	False	False

1.6 Data quality & class balance

In [3]:
```
print(f"Input features : {len(config.FEATURES)}  ({len(config.NUMERIC_FEATURES)} numeric +
{len(config.CATEGORICAL_FEATURES)} categorical)")
print(f"Instances      : {len(raw):,}")
print(f"Missing values : {int(raw.isna().sum().sum())}")
print(f"Duplicate rows : {int(raw.duplicated().sum())}")

class_summary = (raw[config.TARGET].value_counts().rename("count").to_frame()
                 .assign(percent=lambda t: (100 * t["count"] / t["count"].sum()).round(2)))
class_summary.index = class_summary.index.map({False: "0 = No purchase", True: "1 = Purchase"})
class_summary
```

Out [3]:
```
Input features : 17  (10 numeric + 7 categorical)
Instances      : 12,330
Missing values : 0
Duplicate rows : 125
```

Revenue	count	percent
0 = No purchase	10422	84.53
1 = Purchase	1908	15.47

In [4]:
```
raw[config.NUMERIC_FEATURES].describe().T.round(3)
```

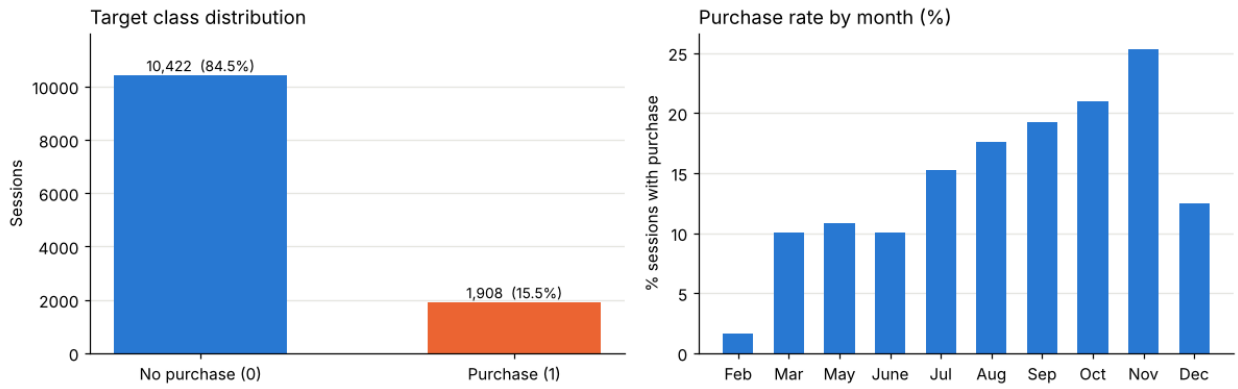
Out [4]:

	count	mean	std	min	25%	50%	75%	max
Administrative	12330.0	2.315	3.322	0.0	0.000	1.000	4.000	27.000
Administrative_Duration	12330.0	80.819	176.779	0.0	0.000	7.500	93.256	3398.750
Informational	12330.0	0.504	1.270	0.0	0.000	0.000	0.000	24.000
Informational_Duration	12330.0	34.472	140.749	0.0	0.000	0.000	0.000	2549.375
ProductRelated	12330.0	31.731	44.476	0.0	7.000	18.000	38.000	705.000
ProductRelated_Duration	12330.0	1194.746	1913.669	0.0	184.138	598.937	1464.157	63973.522
BounceRates	12330.0	0.022	0.048	0.0	0.000	0.003	0.017	0.200
ExitRates	12330.0	0.043	0.049	0.0	0.014	0.025	0.050	0.200
PageValues	12330.0	5.889	18.568	0.0	0.000	0.000	0.000	361.764
SpecialDay	12330.0	0.061	0.199	0.0	0.000	0.000	0.000	1.000

In [5]:
```
fig, axes = plt.subplots(1, 2, figsize=(11, 3.6))

counts = raw[config.TARGET].value_counts().sort_index()
bars = axes[0].bar(["No purchase (0)", "Purchase (1)"], counts.values,
                   color=viz.SERIES_COLORS[:2], width=0.55)
for bar, value in zip(bars, counts.values):
    axes[0].text(bar.get_x() + bar.get_width() / 2, value + 150,
                 f"{value:,}  ({value / counts.sum():.1%})", ha="center", fontsize=9)
axes[0].set_title("Target class distribution", loc="left")
axes[0].set_ylabel("Sessions"); axes[0].set_ylim(0, counts.max() * 1.15); axes[0].grid(axis="x",
visible=False)

month_order = ["Feb", "Mar", "May", "June", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]
purchase_rate = raw.groupby("Month")[config.TARGET].mean().reindex(month_order) * 100
axes[1].bar(purchase_rate.index, purchase_rate.values, color=viz.SERIES_COLORS[0], width=0.6)
axes[1].set_title("Purchase rate by month (%)", loc="left"); axes[1].set_ylabel("% sessions with
purchase")
axes[1].grid(axis="x", visible=False)
plt.tight_layout(); plt.show()
```



Observations from the data * No missing values. There are 125 exact duplicate rows (mostly single-page "bounce" sessions); they are dropped so the same row can't end up in both the train and test sets. * The classes are **imbalanced (≈ 84.5 % / 15.5 %)**, so accuracy alone is misleading: a model that always predicts "no purchase" already gets ≈ 84.5 %. That's why **F1, MCC and AUC** are the main comparison metrics, and tuning optimises **F1 of the purchase class**. * Count and duration features are heavily right-skewed (e.g. `ProductRelated_Duration` ranges up to 63,974 s), so a `log1p` transform is applied before scaling. * `OperatingSystems`, `Browser`, `Region` and `TrafficType` are numeric **codes** for categories (not quantities), so they are one-hot encoded rather than scaled.

1.7 Pre-processing (`src/data.py`, `src/preprocessing.py`)

Step	Applied to	Choice	Why
Remove duplicates	all rows	`drop_duplicates()`	avoid train/test leakage
Target encoding	`Revenue`	True → 1, False → 0	binary label
Skew correction	10 numeric	`log1p`	durations/counts are heavy-tailed
Scaling	10 numeric	`StandardScaler`	needed by distance/margin models (KNN, SVM) and helps LR converge
Encoding	7 categorical	`OneHotEncoder(min_frequency=30, handle_unknown='infrequent_if_exist', drop='if_binary')`	category codes have no order; rare levels are grouped as "infrequent"
Split	—	stratified **80 / 20**, `random_state=42`	keeps the 15.5 % purchase rate in both sets

All of this goes inside a scikit-learn **`Pipeline`**, so the scaler and encoder are fitted **only on training folds**, with no data leakage during cross-validation. Every saved model file therefore carries its own fitted preprocessing.

In [6]:
```
preparer = DataPreparer()
train_raw, test_raw = split_train_test(remove_duplicates(raw))
X_train, y_train = preparer.prepare(train_raw)

print(f"After removing duplicates: {len(train_raw) + len(test_raw):,} rows")
print(f"Train: {len(train_raw):,} rows  purchase rate = {y_train.mean():.3%}")
print(f"Test : {len(test_raw):,} rows  purchase rate =
{preparer.prepare_target(test_raw).mean():.3%}")

encoded_columns = build_preprocessor().fit(X_train).get_feature_names_out()
print(f"17 raw features -> {len(encoded_columns)} model-ready columns after encoding")
```

```
After removing duplicates: 12,205 rows
Train: 9,764 rows  purchase rate = 15.629%
Test : 2,441 rows  purchase rate = 15.649%
17 raw features -> 65 model-ready columns after encoding
```

Step 2: Classification Models & Evaluation Metrics

2.1 Models and hyper-parameter search (`src/models.py`, `src/training.py`)

Each model = `preprocessor` + classifier in one pipeline. Hyper-parameters are chosen with **`GridSearchCV`** **(5-fold stratified CV on the training set only, scoring = F1)**; the test set is used **once**, for the final evaluation.

#	Model	Hyper-parameters searched	Rationale
1	Logistic Regression	`C` ∈ {0.001, 0.003, 0.01, 0.1, 1, 10, 100}, `class_weight` ∈ {None, balanced}	`C` = inverse L2 strength; `balanced` re-weights the minority class
2	Decision Tree	`max_depth` ∈ {3,4,5,6,8,10,None}, `min_samples_leaf` ∈ {1,10,30,60,100,150}, `criterion` ∈ {gini, entropy}, `class_weight` ∈ {None, balanced}	depth & leaf size control over-fitting
3	K-Nearest Neighbors	`n_neighbors` ∈ {5,9,15,25,35,51}, `weights` ∈ {uniform, distance}, `p` ∈ {1 (Manhattan), 2 (Euclidean)}	k trades variance vs bias; features are scaled first
4	Naive Bayes (Gaussian)	`var_smoothing` ∈ 10^{-9} … 10^{2} (12 values)	Gaussian NB suits continuous inputs; smoothing stabilises variance estimates
5	SVM (RBF kernel)	`C` ∈ {0.1, 1, 10}, `gamma` ∈ {scale, 0.001, 0.003, 0.01, 0.1}, `class_weight` ∈ {None, balanced}	RBF captures non-linear boundaries; AUC is computed from the SVM `decision_function` score

Why Gaussian NB (not Multinomial)? Multinomial NB needs non-negative count features. After `log1p` + standardisation the numeric features are continuous and can be negative, so Gaussian NB is the right fit.

The grids above are read straight from the model registry:

In [7]:
```
pd.DataFrame([{"Model": spec.name, "Saved as": f"model/{spec.key}.joblib",
               "Search grid": spec.param_grid()} for spec in MODEL_SPECS]).set_index("Model")
```

Out [7]:

Model	Saved as	Search grid
Logistic Regression	model/logistic_regression.joblib	{'C': [0.001, 0.003, 0.01, 0.1, 1, 10, 100], 'class_weight': [None, 'balanced']}
Decision Tree	model/decision_tree.joblib	{'max_depth': [3, 4, 5, 6, 8, 10, None], 'min_samples_leaf': [1, 10, 30, 60, 100, 150], 'criterion': ['gini', 'entropy'], 'class_weight': [None, 'balanced']}
KNN	model/knn.joblib	{'n_neighbors': [5, 9, 15, 25, 35, 51], 'weights': ['uniform', 'distance'], 'p': [1, 2]}
Naive Bayes (Gaussian)	model/naive_bayes.joblib	{'var_smoothing': [1e-09, 1e-08, 1e-07, 1e-06, 9.999999999999999e-06, 0.0001, 0.001, 0.01, 0.1, 1.0, 10.0, 100.0]}
SVM	model/svm.joblib	{'C': [0.1, 1, 10], 'gamma': ['scale', 0.001, 0.003, 0.01, 0.1], 'class_weight': [None, 'balanced']}

2.2 Train & tune all five models

In [8]:
```
workflow = TrainingWorkflow(preparer=preparer, tuner=HyperparameterTuner(),
evaluator=ModelEvaluator())
result = workflow.run(raw)
result.tuning_summary()
```

Out [8]:
```
Rows after de-duplication: 12205  (train 9764 / test 2441)
Logistic Regression      tuned in    4.5s  best={'C': 0.003, 'class_weight': 'balanced'}
Decision Tree            tuned in   35.2s  best={'class_weight': None, 'criterion': 'gini',
'max_depth': 6, 'min_samples_leaf': 60}
KNN                      tuned in   32.9s  best={'n_neighbors': 35, 'p': 2, 'weights': 'distance'}
Naive Bayes (Gaussian)   tuned in    2.1s  best={'var_smoothing': 1.0}
SVM                      tuned in  218.2s  best={'C': 10, 'class_weight': 'balanced', 'gamma':
0.001}
```

	Best hyper-parameters	CV F1	CV ROC_AUC	Tuning time (s)
Model				
Logistic Regression	{'C': 0.003, 'class_weight': 'balanced'}	0.6669	0.9119	4.5
Decision Tree	{'class_weight': None, 'criterion': 'gini', 'max_depth': 6, 'min_samples_leaf': 60}	0.6563	0.9219	35.2
KNN	{'n_neighbors': 35, 'p': 2, 'weights': 'distance'}	0.6042	0.9134	32.9
Naive Bayes (Gaussian)	{'var_smoothing': 1.0}	0.5921	0.8699	2.1
SVM	{'C': 10, 'class_weight': 'balanced', 'gamma': 0.001}	0.6669	0.9158	218.2

2.3 Save the models, metadata and test data

In [9]:
```
saved_files = save_artifacts(result, preparer, ModelRepository(config.MODEL_DIR),
config.TEST_DATA_PATH)
print("\n".join(str(path.relative_to(config.PROJECT_ROOT)) for path in saved_files))
```

```
Saved /home/padmesh-kumbhar/online-shoppers-classification/model/logistic_regression.joblib
Saved /home/padmesh-kumbhar/online-shoppers-classification/model/decision_tree.joblib
Saved /home/padmesh-kumbhar/online-shoppers-classification/model/knn.joblib
Saved /home/padmesh-kumbhar/online-shoppers-classification/model/naive_bayes.joblib
Saved /home/padmesh-kumbhar/online-shoppers-classification/model/svm.joblib
Saved /home/padmesh-kumbhar/online-shoppers-classification/model/model_comparison.csv
Saved /home/padmesh-kumbhar/online-shoppers-classification/model/metadata.json
Saved /home/padmesh-kumbhar/online-shoppers-classification/test_data.csv
model/logistic_regression.joblib
model/decision_tree.joblib
model/knn.joblib
model/naive_bayes.joblib
model/svm.joblib
model/model_comparison.csv
model/metadata.json
test_data.csv
```

2.4 Evaluation on the held-out test set

For every model: **Accuracy, AUC, Precision, Recall, F1, MCC** (positive class = *Purchase*).

Metric	Formula / meaning
Accuracy	(TP + TN) / (TP + TN + FP + FN)
AUC	Area under the ROC curve (TPR vs FPR over all thresholds); threshold-independent ranking quality
Precision	TP / (TP + FP): of the sessions predicted as buyers, how many actually bought
Recall	TP / (TP + FN): of the actual buyers, how many were caught
F1	2 · Precision · Recall / (Precision + Recall)
MCC	(TP·TN − FP·FN) / √((TP+FP)(TP+FN)(TN+FP)(TN+FN)); ranges from −1 to +1 and stays reliable under class imbalance

In [10]:
```
result.comparison
```

Out [10]:

	Accuracy	AUC	Precision	Recall	F1	MCC
ML Model						
Logistic Regression	0.8808	0.9205	0.5880	0.7958	0.6763	0.6157
Decision Tree	0.9021	0.9262	0.7227	0.6073	0.6600	0.6065
KNN	0.8939	0.9154	0.7043	0.5550	0.6208	0.5655
Naive Bayes (Gaussian)	0.8578	0.8734	0.5347	0.7068	0.6088	0.5316
SVM	0.8820	0.9255	0.5914	0.7958	0.6786	0.6183

In [11]:

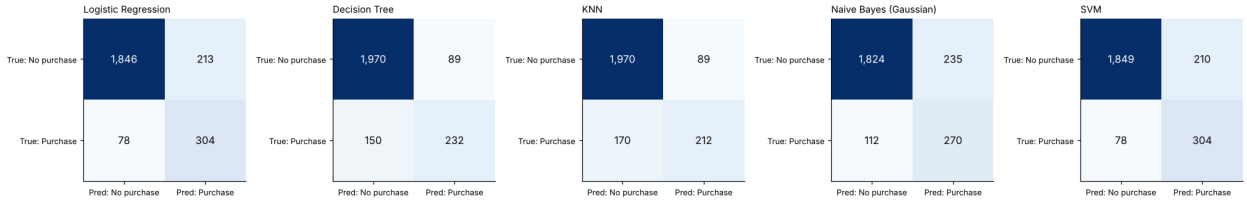
```
print("Best model per metric")
for metric in result.comparison.columns:
    column = result.comparison[metric]
    winners = column.index[column == column.max()].tolist()
    print(f"  {metric:<10}: {' & '.join(winners):<38} ({column.max():.4f})")
```

```
Best model per metric
  Accuracy  : Decision Tree                       (0.9021)
  AUC       : Decision Tree                       (0.9262)
  Precision : Decision Tree                       (0.7227)
  Recall    : Logistic Regression & SVM           (0.7958)
  F1        : SVM                                 (0.6786)
  MCC       : SVM                                 (0.6183)
```

2.5 Confusion matrices (test set)

In [12]:

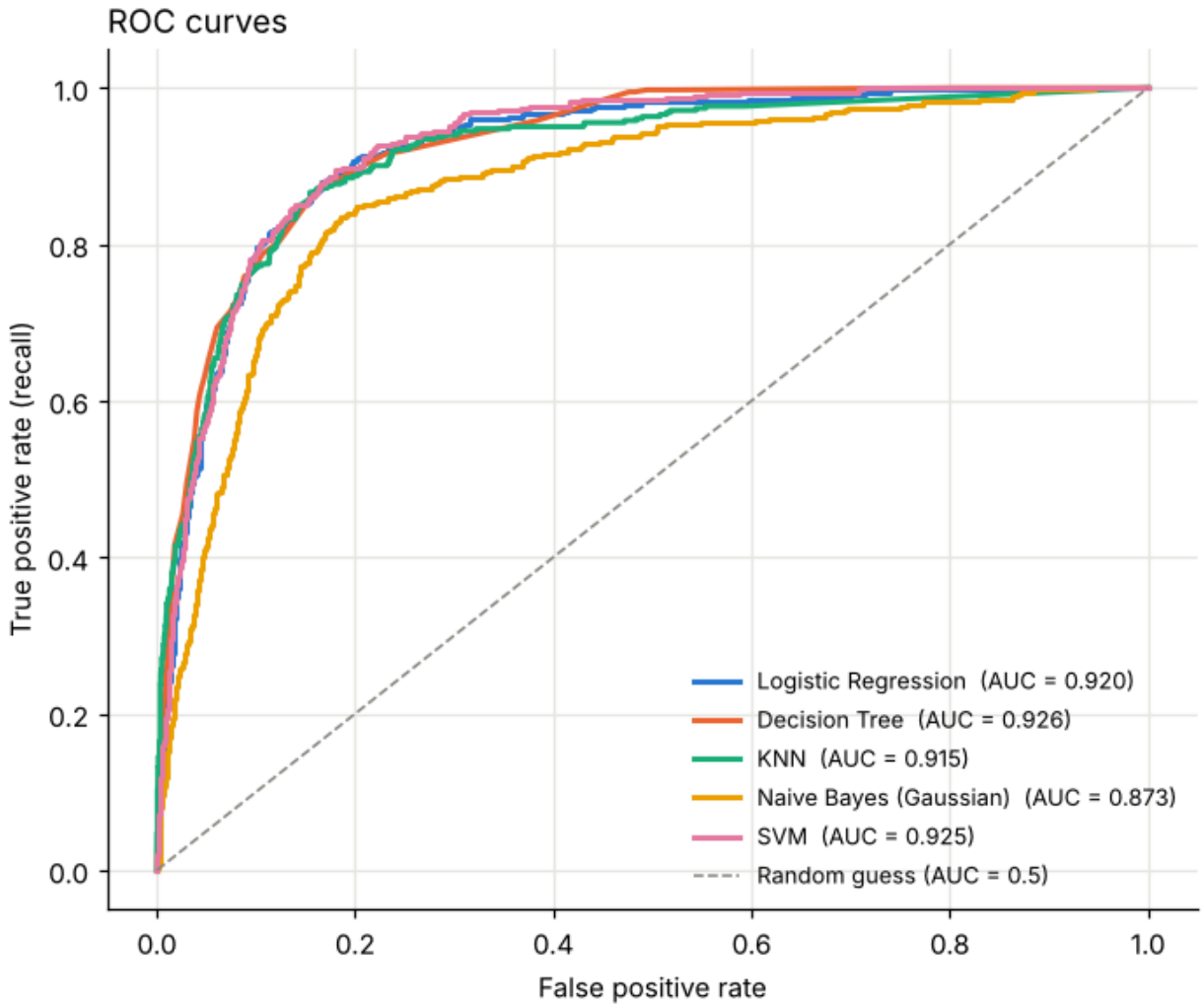
```
viz.plot_confusion_matrices({name: ev.confusion for name, ev in result.evaluations.items()})
plt.show()
```



2.6 ROC curves (test set)

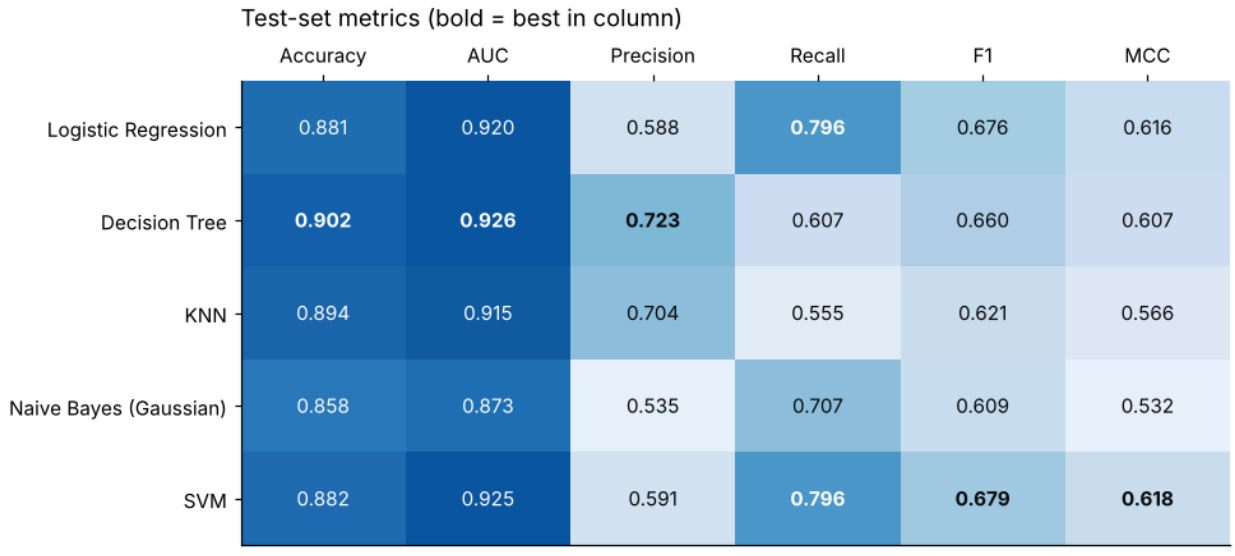
In [13]:

```
viz.plot_roc_curves(result.y_test,
                    scores={name: ev.y_score for name, ev in result.evaluations.items()},
                    auc=result.comparison["AUC"].to_dict())
plt.show()
```



2.7 Metric comparison

In [14]:
```
viz.plot_metric_heatmap(result.comparison)
plt.show()
```

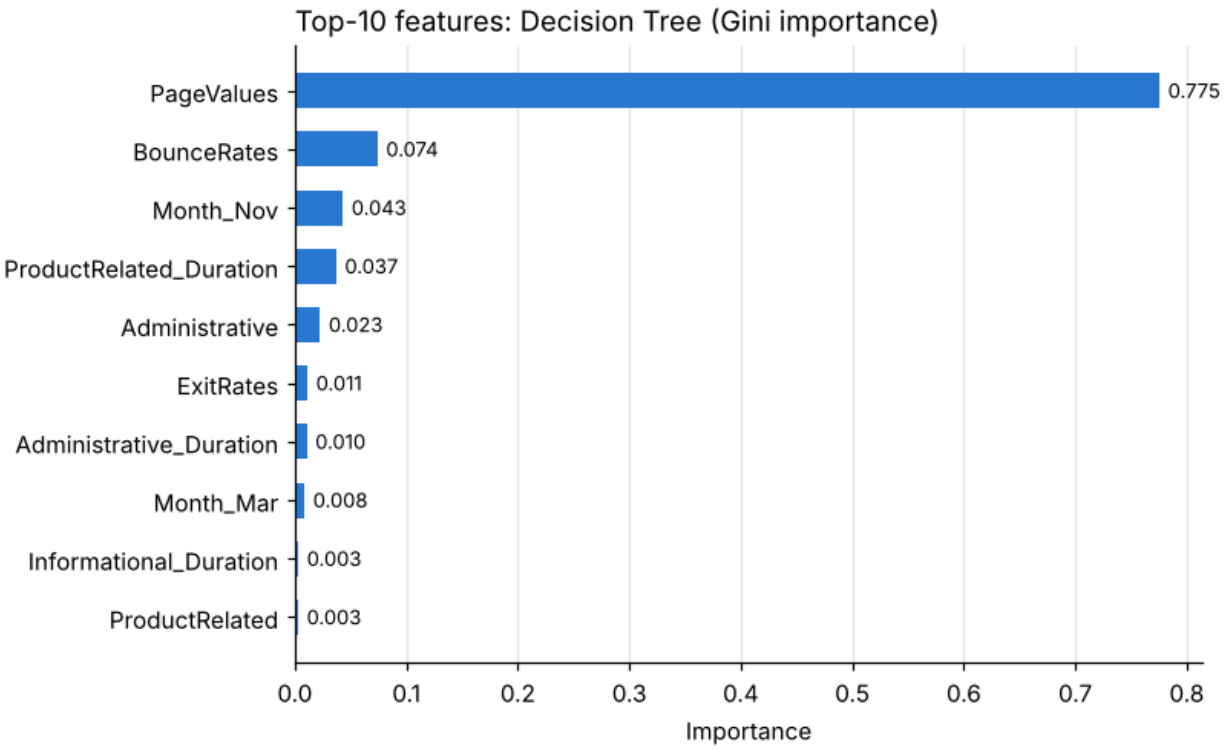


2.8 What drives a purchase? (feature importance from the tuned Decision Tree)

In [15]:

```
tree_pipeline = next(m.pipeline for m in result.trained if m.key == "decision_tree")
feature_names = [name.split("__", 1)[1] for name in
tree_pipeline.named_steps["prep"].get_feature_names_out()]
importance = pd.Series(tree_pipeline.named_steps["clf"].feature_importances_,
index=feature_names)

viz.plot_feature_importance(importance.nlargest(10), "Top-10 features: Decision Tree (Gini
importance)")
plt.show()
```



2.9 Comparison table & observations

Test-set results (2,441 sessions, 382 purchases)

ML Model	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	0.8808	0.9205	0.5880	**0.7958**	0.6763	0.6157
Decision Tree	**0.9021**	**0.9262**	**0.7227**	0.6073	0.6600	0.6065
KNN	0.8939	0.9154	0.7043	0.5550	0.6208	0.5655
Naive Bayes (Gaussian)	0.8578	0.8734	0.5347	0.7068	0.6088	0.5316
SVM	0.8820	0.9255	0.5914	**0.7958**	**0.6786**	**0.6183**

ML Model	Observation on performance
Logistic Regression	Strong regularisation (`C = 0.003`) with `class_weight='balanced'` . It catches **~80 % of buyers** (recall 0.796, 304 of 382) at precision 0.588, and its F1/MCC are almost the same as SVM's. Once the features are log-scaled the class boundary is close to linear, and `PageValues` carries most of the signal (largest coefficient). Tuning takes ~4 s and the model is interpretable, so it's the best value-for-cost model.
Decision Tree	Shallow tree (`max_depth = 6` , `min_samples_leaf = 60` , no class weighting). **Highest Accuracy (0.902), AUC (0.926) and Precision (0.723)**: when it predicts "purchase" it's right 72 % of the time. Because it isn't re-weighted, it leans towards the majority class: recall is only 0.607 (misses 150 buyers). The first split is on `PageValues` (importance 0.78), followed by `BounceRates` and `Month = Nov` , which gives simple, explainable rules.
KNN	`k = 35` , distance-weighted, Euclidean. Good precision (0.704) but the **lowest recall (0.555)**: in an 84/16 dataset most neighbours of a buyer are non-buyers, and KNN has no class-weight option. It also keeps all 9,764 training rows in memory and is slow at prediction time.
Naive Bayes (Gaussian)	Needed very heavy smoothing (`var_smoothing = 1.0`) and is **weakest on Accuracy (0.858), AUC (0.873) and MCC (0.532)**. Its two assumptions are both violated here: features are strongly correlated (`BounceRates` – `ExitRates` r = 0.91, `ProductRelated` – `ProductRelated_Duration` r = 0.86), and one-hot features aren't Gaussian. It still gets a reasonable recall (0.707) and is the fastest model to train.
SVM (RBF)	`C = 10` , `gamma = 0.001` , `class_weight='balanced'` . **Best F1 (0.679) and MCC (0.618)**, joint-best recall (0.796) and AUC 0.9255, practically tied with the Decision Tree (0.9262). The small `gamma` gives a smooth, nearly linear boundary, which is why it performs close to Logistic Regression. It is the most expensive model: ~190 s of tuning vs ~4 s for LR.

Conclusion

- **Accuracy alone is misleading here.** Every model scores 0.86–0.90, close to the 0.845 you'd get by always predicting "no purchase". **MCC and F1** separate the models much better.
- **Best overall: SVM** (highest MCC & F1), with **Logistic Regression a near-equal and far cheaper alternative**.
- **Pick by business goal:**
- To catch as many buyers as possible (cheap nudges such as a pop-up or chat), use **SVM / Logistic Regression** (≈ 80 % recall).
- To target only very likely buyers (costly incentives such as discounts), use the **Decision Tree** (72 % precision).
- `class_weight='balanced'` shifts the precision–recall trade-off: weighted models (LR, SVM) gain recall, unweighted ones (DT, KNN) gain precision.
- **`PageValues` is the dominant predictor**: 56 % of sessions with `PageValues > 0` end in a purchase, vs only 4 % when it is 0. **November** (pre-holiday season) is the strongest month effect.